



भारतीय सूचना प्रौद्योगिकी संस्थान गुवाहाटी
Indian Institute of Information Technology Guwahati

CS236 - ARTIFICIAL INTELLIGENCE LAB
ASSIGNMENT - 04

Problem Statement

Write a Python program to solve the $(n^2 - 1)$ -puzzle problem using the following search algorithms:

- **Breadth-First Search (BFS)**
- **Depth-First Search (DFS)** — *Implemented as Tree Search*
- **Depth-Limited Search (DLS)** — *Use a depth limit of $L = 20$*
- **Iterative Deepening (ID)**

The program should:

1. Accept n as input ($n = 3$ for 8-puzzle, $n = 4$ for 15-puzzle).
2. Generate **10 random initial states**. Do not check for solvability; if a state is unsolvable, the algorithm should eventually exhaust the search space or timeout.
3. Set a **strict 1-minute timeout** per algorithm per initial state.
4. Categorize results into three outcomes: **Success**, **Failure (Exhausted)** (search finished but no solution found), or **Timeout** (1 minute reached).

Experimental Requirements

1. Constraints on Environment

To ensure you understand the core logic and syntax of the algorithms, you **must not** use an IDE or advanced code editor.

- Use a basic text editor (e.g., Notepad, Gedit, Vim) that **does not** support syntax highlighting, code autocompletion, auto-indentation, or AI-assisted coding (Copilot, etc.).
- Compilation/Execution must be done via the command line.

2. Data Collection & Plotting

For each value of $n \in \{3, 4\}$, generate the following plots:

- **Average Execution Time:** Compare the four algorithms. (Note: For timeouts, record the time as 60s).
- **Success Rate:** A bar chart showing the percentage of the 10 problems solved by each algorithm.
- **Average Path Length:** For successful runs, compare the number of moves in the solution found by BFS, DFS, DLS, and ID.

Key Technical Observations

Students should pay close attention to the following behaviors during execution:

- **DFS (Tree Search):** Notice what happens when DFS encounters a state it has seen before. Because it is a tree search, it does not keep track of "visited" states. How does this affect the 1-minute timeout?
- **Scalability ($n = 3$ vs $n = 4$):** Observe the success rate as n increases. Does any algorithm successfully solve a 15-puzzle within 1 minute using uninformed search?
- **Path Optimality:** Compare the path length of BFS vs DFS. Does DFS find the shortest path?
- **Solvability:** For $n = 3$, if an algorithm exhausts the entire state space without finding a goal, what does that imply about the initial state configuration?

Submission Requirements

- **Source Code:** A single Python file implementing all four algorithms.
- **Report:** A PDF including:
 1. The three required plots (Time, Success Rate, Path Length).
 2. A brief discussion on why DFS Tree Search behaves differently from BFS in this specific puzzle.
 3. An explanation of what happened to the success rate when moving from $n = 3$ to $n = 4$.